

If there is a critical section of code that can be executed by only one thread at a time, a mutex is associated with the critical section. Before a thread enters the critical section, it needs to acquire the lock associated with that critical section. If the lock is already held by another thread, then the current thread needs to wait until the currently executing thread releases the lock.

Let T1 be the thread requesting the lock and let T2 be the thread currently holding the lock. What should T1 do, when it finds that the lock is not available? Should it keep checking the lock variable until it becomes free, or should it just put itself to sleep for a short duration and then come back and check whether the lock has been released? Depending on the nature of the application, either option can be beneficial.

A spinlock is a lock in which the thread simply waits ('spins') until the lock becomes available. It is very efficient if the threads are only likely to be blocked for a short period of time, as it avoids the overhead of operating system process re-scheduling.

It is wasteful if the lock is held for a long period of time. It is also possible for the thread to spin for a short duration on the lock and if the lock is not released during that duration, it will go to sleep and get rescheduled by the OS. Consider the following code snippet where the thread continues to spin on the lock until it acquires the lock:

```
pthread_mutex_t mutex1;
int all_things_done;

void my_thread_work(void)
{
    RETRY:  if (pthread_mutex_lock(&mutex1))
    {
        goto RETRY;;
    }
    Things_done++;
    Pthread_mutex_unlock(&mutex1);
}
```

This is an example for a pure spinlocking code, where the thread requesting the lock continues to spin on the CPU until it gets the lock or until its CPU quantum is complete. If the critical section protected by the spinlock is long, then the requesting thread unnecessarily ends up wasting CPU cycles by spinning.

On the other hand, if the critical section protected by the spinlock is quite short, then the above code snippet is preferable, since the thread can acquire the lock instead of having to be rescheduled by the OS.

Depending on the nature of the critical section, a programmer can decide to use a pure spinlock or a lock, which spins for a short duration and then goes off to sleep if the lock is not obtained during the spin duration.

```
pthread_mutex_t mutex1;
int all_things_done;
void my_thread_work(void)
{
    RETRY:  if (pthread_mutex_lock(&mutex1))
    {
        retry_Count++;
        If (retry_count < MAX_RETRY_COUNT)
            goto RETRY;;
        sleep(SLEEP_DUR);
        goto RETRY;
    }
    Things_done++;
    Pthread_mutex_unlock(&mutex1);
}
```

In the above code snippet, the requesting thread spins until the number of tries it makes to acquire the lock reaches the MAX_RETRY_COUNT. After that, it goes to sleep and can be rescheduled by the OS. The values of MAX_RETRY_COUNT and SLEEP_DUR are, typically, application and critical section dependent.

Lock granularity

An important property of a lock is its granularity. The granularity is a measure of the amount of data the lock is protecting. In general, choosing a coarse granularity (a lock protecting a large segment of data) results in less lock overhead when few threads are accessing the protected data, but worse performance when multiple threads are running concurrently and accessing different parts of the protected data. This is because of increased lock contention.

Lock contention occurs whenever one thread attempts to acquire a lock held by another thread. The more granular the available locks, the less likely it is that one thread will request a lock held by the other, since the data areas they are accessing are different.

The more coarse the lock, the higher the likelihood that the lock will stop an unrelated thread from proceeding. Conversely, using a fine granularity (a larger number of locks, each protecting a fairly small amount of data) increases the overhead of the locks themselves but reduces lock contention.

More locks also increase the risk of deadlock. Consider the following code snippet, which shows the skeleton code for ticket booking in a cinema theatre. The two-dimensional matrix 'tickets' represents the tickets available in a cinema hall. The seating in the cinema hall consists of rows and columns. A ticket is represented by a unique tuple of (row, column).

Ticket booking is done by a multi-threaded application, with each thread representing a booking window counter. The cinema hall has multiple booking windows. Hence, access to the matrix 'tickets' needs to be protected by a lock and performed inside